

Arquitectura DDD

Multiplex

Elaborado por:

Hugo Alejandro Hernández Mercado

Juan José Espinosa Alzate

Santiago Viana Ayala

Cesar Augusto López Gallego

Arquitectura de Software

Universidad Pontificia Bolivariana

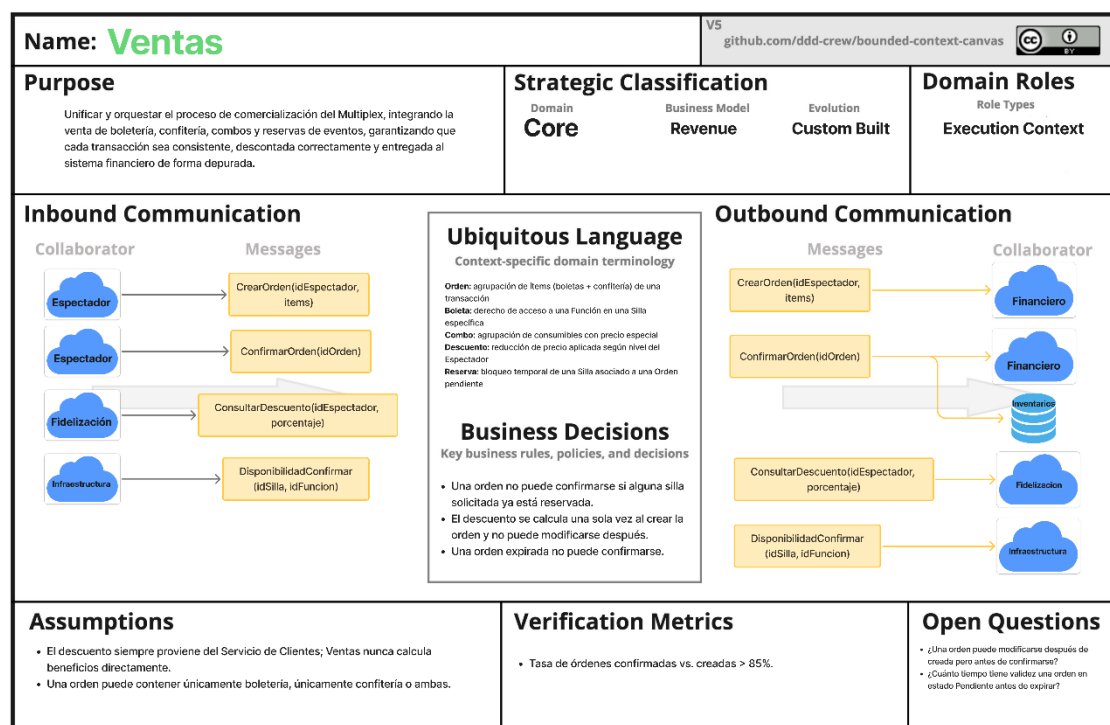
Medellín, 5 de mayo de 2026

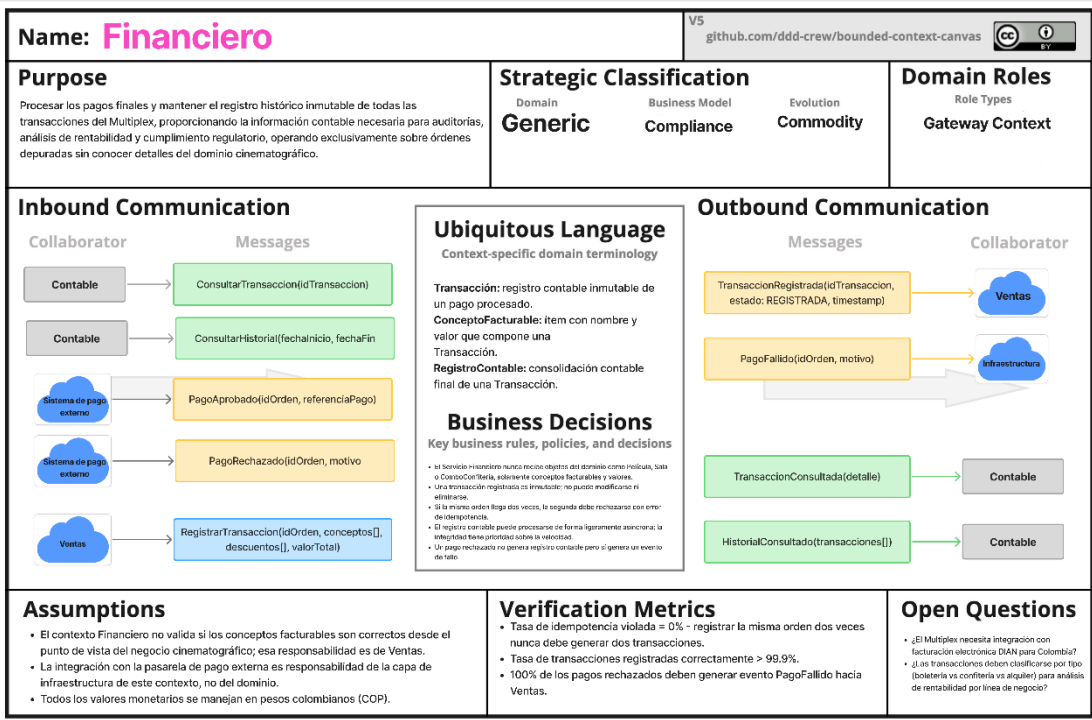
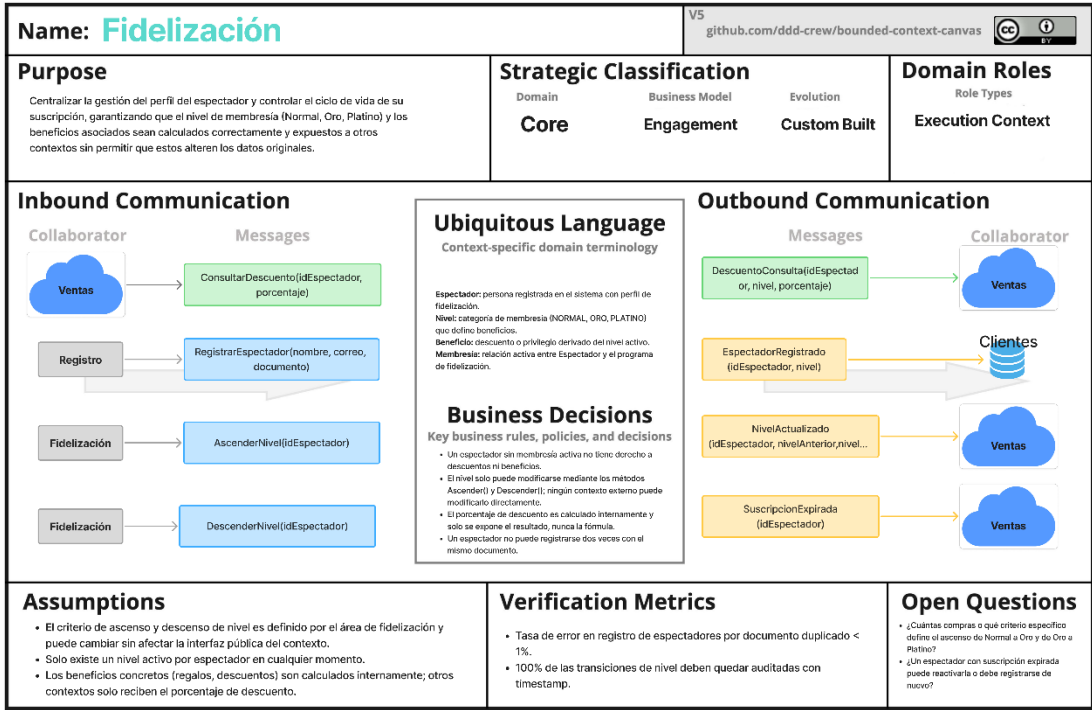
IDENTIFICACIÓN DEL DOMINIO Y SUBDOMINIOS

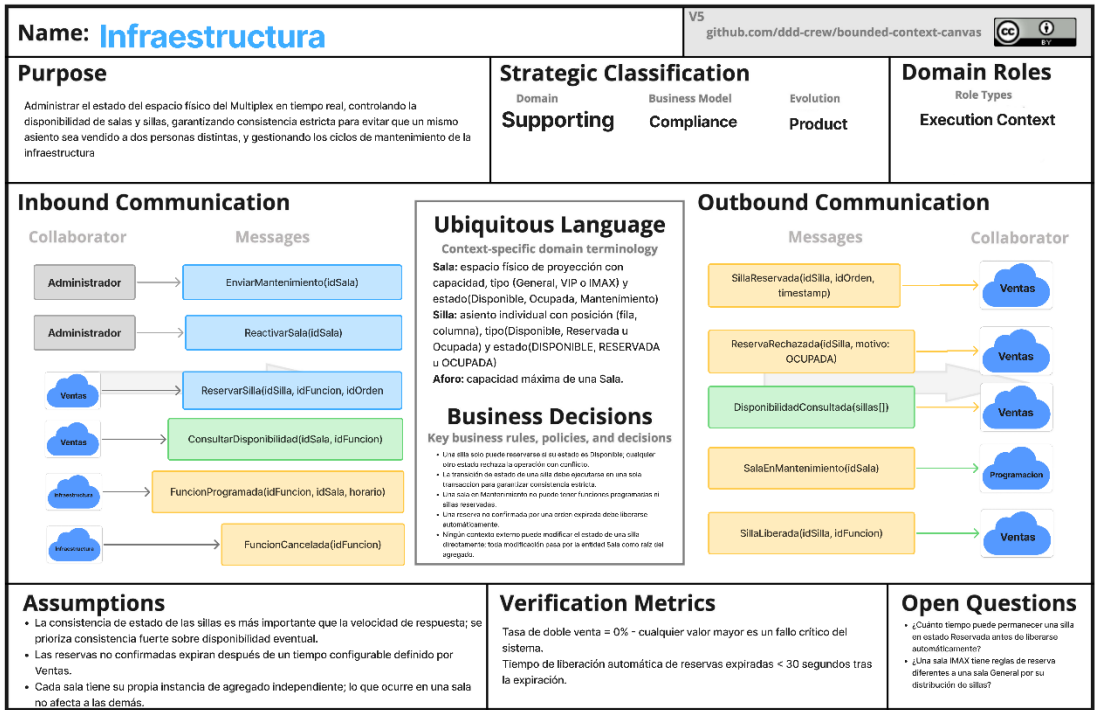
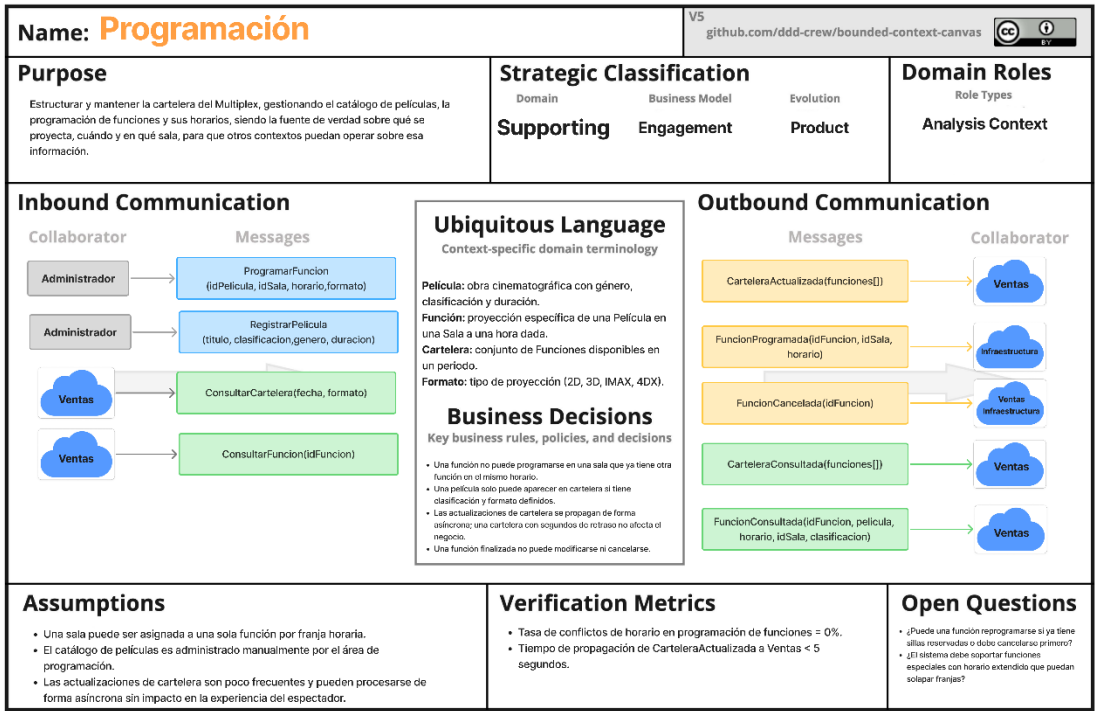
Subdominio	Tipo	Justificación
Ventas	Core	Diferenciador competitivo; aquí está el valor
Clientes y Fidelización	Core	El programa de niveles (Normal/Oro/Platino) es ventaja competitiva
Programación	Supporting	Necesario pero estándar; define cartelera y funciones
Infraestructura Física	Supporting	Gestión de salas y sillas; crítico en consistencia pero no diferenciador
Financiero	Generic	Contabilidad y pagos; podría externalizarse
Cadena	Generic	Configuración corporativa; bajo cambio, bajo riesgo

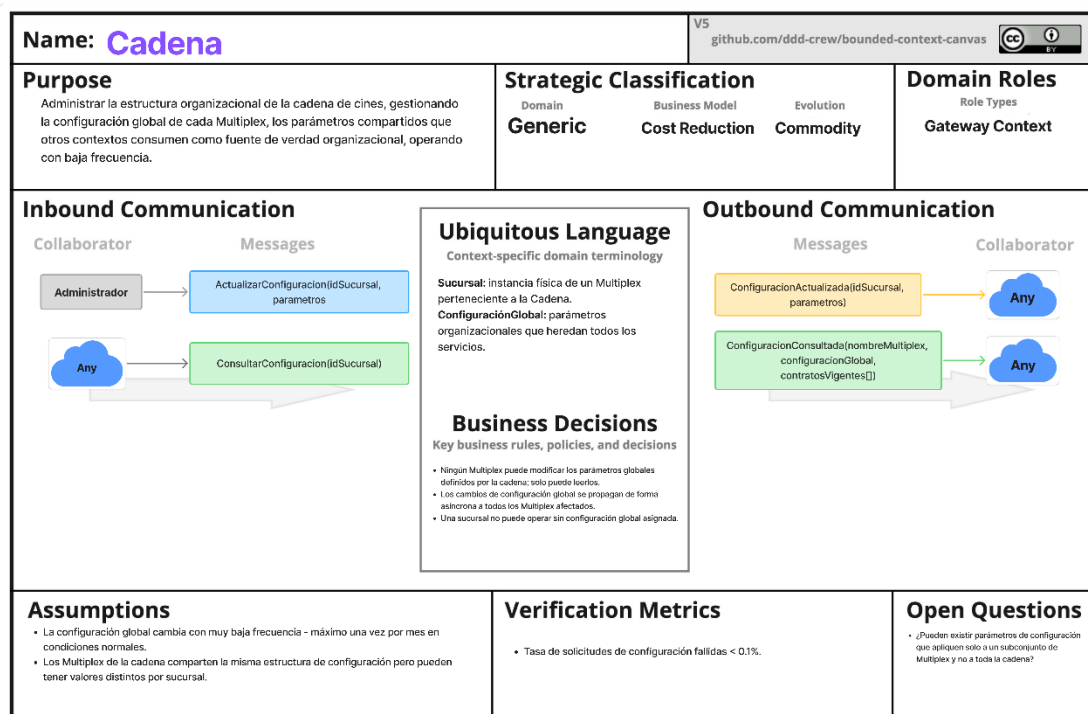
<https://www.figma.com/board/sNIEWB1BmC4sDAo6f6Koh/Multiplex-DDD?node-id=0-1&t=2QQ82l6dEGdyl5sU-1>

BOUNDED CONTEXT









LENGUAJE UBICUO POR CONTEXTO

BC Clientes:

- Espectador: persona registrada en el sistema con perfil de fidelización.
- Nivel: comportamiento de membresía encapsulado en una clase concreta (NivelNormal, NivelOro, NivelPlatino) que implementa INivel y define su propio cálculo de descuento y beneficios
- Beneficio: descuento o privilegio derivado del nivel activo.
- Membresía: relación activa entre Espectador y el programa de fidelización.

BC Programación:

- Película: obra cinematográfica con género, clasificación y duración.
- Función: proyección específica de una Película en una Sala a una hora dada.
- Cartelera: conjunto de Funciones disponibles en un periodo.
- Formato: comportamiento de proyección encapsulado en una clase concreta (Formato2D, Formato3D, FormatoIMAX, Formato4DX) que implementa IFormatoProyeccion y define su propia compatibilidad con tipos de sala y precio extra.
- Alquiler: reserva exclusiva de una sala para un evento privado o proyección corporativa en un rango horario definido.

BC Infraestructura:

- Sala: espacio físico de proyección con capacidad, tipo y estado.
- Silla: asiento individual con posición (fila, columna) y estado de disponibilidad.
- Aforo: capacidad máxima de una Sala.
- Estado de Silla: DISPONIBLE, RESERVADA u OCUPADA.

BC Ventas:

- Orden: agrupación de ítems (boletas + confitería) de una transacción.
- Boleta: derecho de acceso a una Función en una Silla específica.
- Combo: agrupación de consumibles con precio especial definida en DefCombo.
- Descuento: reducción de precio aplicada según nivel del Espectador.
- Reserva: bloqueo temporal de una Silla asociado a una Orden pendiente.

BC Financiero:

- Transacción: registro contable inmutable de un pago procesado.
- ConceptoFacturable: ítem con nombre y valor que compone una Transacción.
- RegistroContable: consolidación contable final de una Transacción.

BC Cadena:

- Sucursal: instancia física de un Multiplex perteneciente a la Cadena.
- ContratoCorporativo: acuerdo de servicio con terceros.
- ConfiguraciónGlobal:
- conjunto de ParametrosGlobales tipados que los demás contextos consultan como fuente de verdad organizacional. Cada parámetro es un Value Object ParametroGlobal con clave, valor y tipo definidos. Ningún contexto externo puede modificarlos directamente.
- IEstadoContrato: comportamiento del ciclo de vida de un ContratoCorporativo encapsulado en clases concretas (ContratoVigente, ContratoVencido, ContratoCancelado) que definen si el contrato puede ejecutarse o no.

AGREGADOS, ENTIDADES Y OBJETOS DE VALOR

BC: Clientes y Fidelización

Agregados y Entidades

Espectador es el Agregado Raíz de este contexto. Representa a la persona registrada en el sistema con un perfil de fidelización activo. Es dueño exclusivo de sus datos; ningún

otro servicio puede modificarlos directamente. Suscripcion es una entidad anidada dentro de Espectador que gestiona el ciclo de vida de la membresía y delega el comportamiento de estado y nivel mediante los patrones State y Strategy.

Entidad	Atributos clave	Tipo en DDD
Espectador	id, nombre, correo, documento, suscripcion	AggregateRoot
Suscripcion	estado: IEstadoSuscripcion, nivel: INivel, fechaInicio, fechaFin	Entity (en Espectador)

Value Objects del BC:

Value Object	Definición y restricciones
EspectadorId	UUID inmutable generado en el momento del registro. No puede reasignarse.
Email	Dirección de correo validada según RFC 5321. Inmutable tras el registro. No pueden existir dos Espectadores con el mismo Email.
Documento	Tipo (CC, CE, PAS) + número. Único en todo el sistema. Inmutable tras el registro.
NombreCompleto	Nombre + apellido. Inmutable. Longitud mínima 2 caracteres por campo.
INivel	Se aplica patron strategy para manejar los niveles disponibles. Garantiza que no existan niveles inválidos en el sistema.
IEstadoSuscripcion	Se aplica patron state para manejar el estado de la suscripcion ya sea activa o cancelada
PorcentajeDescuento	Decimal entre 0.0 y 1.0 inclusive. Calculado internamente a partir del NivelSuscripcion. Solo se expone el resultado, nunca la fórmula.

BeneficioActivo	Descripcion (string) + tipo (DESCUENTO, ACCESO, REGALO) + vigencia (fechaInicio, fechaFin). Encapsula el beneficio concreto derivado del nivel.
-----------------	---

Patrones aplicados:

- **Strategy:** INivel con NivelNormal, NivelOro, NivelPlatino.
- **State:** IEstadoSuscripcion con SuscripcionActiva, SuscripcionExpirada, SuscripcionCancelada.

Principios SOLID aplicados:

- **SRP:** Suscripcion gestiona el ciclo de vida; INivel gestiona el comportamiento de descuento.
- **OCP:** agregar NivelDiamante es crear NivelDiamante implements INivel sin modificar nada.
- **LSP:** cualquier INivel es intercambiable en Suscripcion sin romper su comportamiento.
- **DIP:** Suscripcion depende de INivel e IEstadoSuscripcion, nunca de clases concretas.

BC: Programación

Este BC tiene tres Agregados Raíz independientes. Funcion representa una proyección específica de una Pelicula en una Sala. No guarda el objeto Sala completo; solo una referencia SalaRef con el ID, respetando los límites del contexto. Pelicula es un Agregado Raíz independiente no una entidad dentro de Funcion porque la misma Pelicula puede aparecer en multiples funciones y necesita identidad propia. Cartelera agrupa referencias a Funciones disponibles en un periodo y dispara eventos ante cambios (patron Observer).

Entidad	Atributos clave	Tipo en DDD
Funcion	id, peliculaRef, horario, salaRef, formato, estado	AggregateRoot
Pelicula	id, titulo, clasificacion, genero, duracion	AggregateRoot independiente

Cartelera	id, periodo, funciones[], observadores: IObserverCartelera[]	AggregateRoot
Alquiler	id, salaRef, rangoHorario, solicitante, propósito, estado, capacidadReservada	AggregateRoot

Value Objects del BC:

Value Object	Definición y restricciones
PeliculaRef	Solo el UUID de la pelicula. Referencia cruzada dentro del mismo BC. No contiene atributos de Pelicula.
SalaRef	Solo el UUID de la sala (referencia cruzada al BC Infraestructura). No contiene atributos de Sala.
RangoHorario	Inicio (DateTime) + fin (DateTime). Invariante: inicio < fin. Valida solapamiento con otras funciones en la misma sala.
Clasificacion	Enum cerrado: G, PG, PG13, R. Evita strings libres que generen inconsistencias en la cartelera.
Duracion	Entero positivo en minutos. Inmutable. Restriccion: valor > 0 y valor <= 300.
PeriodoCartelera	FechaInicio + fechaFin de la vigencia de la cartelera. Invariante: inicio < fin.
AlquilerId	UUID inmutable
PropósitoAlquiler	Enum: PROYECCION_PRIVADA, EVENTO_CORPORATIVO, CUMPLEAÑOS, OTRO
EstadoAlquiler	

--	--

Interfaz de comportamiento:

Elemento	Tipo	Responsabilidad
IFormatoProyeccion	Interfaz	getNombre(), esCompatibleConSala(ITipoSala), getPrecioExtra(): Dinero
Formato2D	Clase	Sin precio extra. Compatible con SalaGeneral y SalaVip
Formato3D	Clase	Precio extra definido. Compatible con SalaGeneral, SalaVip e Salalmax
FormatoIMAX	Clase	Precio extra mayor. Solo compatible con Salalmax
Formato4DX	Clase	Precio extra mayor. Compatible con salas especiales

Elemento	Tipo	Responsabilidad
IEstadoFuncion	Interfaz	iniciar(), finalizar(), cancelar(motivo), esModificable()
FuncionProgramada	Clase	Permite iniciar() y cancelar(). esModificable() = true
FuncionEnCurso	Clase	Solo permite finalizar(). esModificable() = false
FuncionFinalizada	Clase	Rechaza cualquier operación. esModificable() = false
FuncionCancelada	Clase	Rechaza cualquier operación. esModificable() = false

Elemento	Tipo	Responsabilidad
IObserverCartelera	Interfaz	actualizar(evento: CarteleraActualizada): void
Cartelera	Sujeto observable	Mantiene lista de IObservadorCartelera. Notifica via

		IPublisher al detectar cambios
ActualizarCartelera	Observador concreto	Implementa IObservadorCartelera. Reacciona ante CarteleraActualizada actualizando la vista de funciones disponibles en Ventas

Patrones aplicados:

- **Strategy** - IFormatoProyeccion con Formato2D, Formato3D, FormatoIMAX, Formato4DX.
- **State** - IEstadoFuncion con FuncionProgramada, FuncionEnCurso, FuncionFinalizada, FuncionCancelada.
- **Observer** – IObservadorCartelera

Principios SOLID aplicados:

- **SRP** - Funcion gestiona la proyección; IFormatoProyeccion gestiona compatibilidad y precio extra.
- **OCP** - agregar Formato4DXPlus es crear una nueva clase sin modificar nada existente.
- **LSP** - cualquier IFormatoProyeccion es intercambiable en Funcion sin romper su comportamiento.
- **DIP** - Funcion depende de IFormatoProyeccion e IEstadoFuncion, nunca de clases concretas.

BC: Infraestructura

Sala es el Agregado Raiz y contiene a las Sillas como entidades internas. Gestiona su propio estado mediante el patrón State y delega el llenado de sillas mediante Strategy. Todo acceso a las sillas pasa obligatoriamente por la Sala ningún contexto externo puede modificar el estado de una silla directamente. Silla es la entidad más volátil del sistema: su estado cambia en tiempo real con cada reserva, ocupación o liberación.

Entidad	Atributos clave	Tipo en DDD
---------	-----------------	-------------

Sala	id, nombre, tipo: ITipoSala, estado: IEstadoSala, aforo, sillas[]	AggregateRoot
Silla	id, posicion, tipo: ITipoSilla, estado: IEstadoSilla, reservaExpiracion n	Entity (en Sala)

Value Objects del BC:

Value Object	Definición y restricciones
Salald	UUID unico por sala. Inmutable.
Sillald	UUID unico por silla. Inmutable.
Posicion	Fila (string) + columna (entero). Inmutable tras la creacion de la silla. Unica dentro de la sala.
Aforo	Capacidad maxima de la sala. Entero positivo. Inmutable. Restriccion: valor >= 1.
ReservaExpiracion	Timestamp de expiracion de la reserva. No puede ser fecha pasada al momento de reservar. Cuando se alcanza, la silla vuelve a DISPONIBLE automaticamente.

Interfaz de comportamiento

Elemento	Tipo	Responsabilidad
ITipoSala	Interfaz	obtenerTipo(), llenarSillas(Silla[]): void, esCompatibleConFormato(IFormatoProyeccion): boolean
SalaGeneral	Clase	Llenado con sillas GENERAL y ESPECIAL. Compatible con Formato2D y Formato3D
SalaVip	Clase	Llenado con sillas VIP y ACOMPANANTE. Compatible con Formato2D y Formato3D
Salalmax	Clase	Llenado con sillas GENERAL y VIP. Compatible con FormatoIMAX y Formato3D

Elemento	Tipo	Responsabilidad
ITipoSilla	interfaz	getPrecioBase(): Dinero, esReservable(): boolean, esCompatibleConTipoSala(ITipoSala): boolean
SillaGeneral	Clase	Precio base estándar. Reservable. Compatible con SalaGeneral e Salalmax
SillaVip	Clase	Precio base mayor. Reservable. Compatible con SalaVip e Salalmax
SillaAcompañante	Clase	Sin precio propio. No reservable independientemente. Compatible con SalaVip

Elemento	Tipo	Responsabilidad
IEstadoSala	Interfaz	iniciarFuncion(), finalizarFuncion(), enviarMantenimiento(), reactivar()
SalaDisponible	Clase	Permite iniciarFuncion() y enviarMantenimiento()
SalaOcupada	Clase	Solo permite finalizarFuncion()
SalaMantenimiento	Clase	Solo permite reactivar()

Elemento	Tipo	Responsabilidad
IEstadoSilla	Interfaz	reservar(), ocupar(), liberar()
SillaDisponible	Clase	Solo permite reservar()
SillaReservada	Clase	Permite ocupar() y liberar()
SillaOcupada	Clase	Solo permite liberar()

Patrones aplicados:

- **Strategy** - ITipoSala, ITipoSilla, IStrategyLlenarSilla.
- **State** - IEstadoSala, IEstadoSilla.

Principios SOLID aplicados:

- **SRP** - Sala gestiona consistencia del agregado; ITipoSala gestiona estrategia de llenado y compatibilidad.
- **OCP** - agregar Salalmax4D es crear una nueva clase sin modificar ITipoSala ni Sala.

- **LSP** - cualquier ITipoSala o ITipoSilla es intercambiable sin romper el comportamiento del agregado.
- **ISP** - ITipoSala e ITipoSilla exponen solo lo necesario para cada responsabilidad.
- **DIP** - Sala depende de ITipoSala, IEstadoSala, nunca de clases concretas.

BC: Ventas

Orden es el Agregado Raiz y el elemento central del negocio. Orquesta en una sola unidad la boletería, la confitería y el descuento aplicado. Es mutable hasta que se confirma o expira; después su estado es inmutable. ItemBoleta representa el derecho a ocupar una Silla específica en una Funcion; es inmutable tras su creación.

ItemConfiteria representa un producto del menú con cantidad y precio unitario; también inmutable. Combo es un Value Object dentro de ItemConfiteria porque no tiene identidad propia más allá de su composición.

Entidad	Atributos clave	Tipo en DDD
Orden	id, espectadorRef, items[], descuento, estado: IEstadoOrden, expiracion	AggregateRoot
ItemBoleta	id, funcionRef, sillaRef, precioBase: Dinero	Entity (en Orden)
ItemConfiteria	id, productoRef, cantidad, precioUnitario: Dinero, combo?	Entity (en Orden)
ProductoConfiteria	id, nombre, descripcion, precioUnitario: Dinero, stock	Entity independiente
DefCombo	id, nombre, consumibles[], precioEspecial: Dinero, activo, regla: IReglaCombo	Entity independiente

Value Objects del BC:

Value Object	Definición y restricciones
--------------	----------------------------

OrdenId	UUID inmutable. También sirve como idempotency key para evitar ordenes duplicadas.
EspectadorRef	Solo el UUID del espectador (referencia cruzada al BC Clientes). No contiene datos del perfil.
FuncionRef	Solo el UUID de la funcion (referencia cruzada al BC Programacion).
SillaRef	Solo el UUID de la silla (referencia cruzada al BC Infraestructura).
Dinero	Monto (decimal ≥ 0) + moneda (COP). Soporta suma y multiplicacion. Inmutable; las operaciones retornan nuevas instancias.
Descuento	Porcentaje (decimal 0.0-1.0) + nivelOrigen (NivelSuscripcion). Calculado una sola vez al crear la Orden. No puede modificarse despues.
Expiracion	Timestamp futuro asignado al crear la Orden. Inmutable. Restriccion: debe ser mayor al momento de creacion.
EstadoOrden	Enum cerrado: PENDIENTE, CONFIRMADA, EXPIRADA, CANCELADA. Las transiciones son unidireccionales desde PENDIENTE.
Combo	Lista de consumibles[] + precioEspecial (Dinero). Value Object dentro de ItemConfiteria. Su igualdad se determina por su composicion, no por un-ID.

Interfaz de comportamiento

Elemento	Tipo	Responsabilidad
IEstadoOrden	Intefaz	confirmar(), cancelar(motivo), expirar(), esModificable()

OrdenPendiente	Clase	Permite confirmar(), cancelar() y expirar(). esModificable() = true
OrdenConfirmada	Clase	Rechaza cualquier transición. esModificable() = false
OrdenExpirada	Clase	Rechaza cualquier transición. esModificable() = false
OrdenCancelada	Clase	Rechaza cualquier transición. esModificable() = false

Elemento	Tipo	Responsabilidad
IPoliticaPrecios	Interfaz	calcularPrecio(ITipoSilla, IFormatoProyeccion): Dinero
PoliticaPreciosEstandar	Clase	Implementa política de precios estándar del Multiplex
PoloticaPreciosPromocion	Clase	Implementa precios con descuentos especiales por temporada

- **State** - IEstadoOrden con OrdenPendiente, OrdenConfirmada, OrdenExpirada, OrdenCancelada.
- **Strategy** - IPoliticaPrecios.
- **Factory Method** - FabricaBoleta con FactoryBFuncion y FactoryBEvento.
- **Abstract Factory** - IConsumibleFactory con FactoryComboPequeno, FactoryComboGrande, FactoryComboFamiliar.
- **Builder** - ComboBuilder construye Combo paso a paso. ComboDirector conoce recetas predefinidas.
- **Composite** - IConsumible unifica elementos simples y Combo como nodo compuesto. Combo contiene IConsumible permitiendo combos dentro de combos.

Principios SOLID aplicados:

- **SRP** - Orden gestiona la transacción; IEstadoOrden gestiona transiciones; IPoliticaPrecios gestiona precios; IReglaCombo gestiona validación comercial de combos.

- **OCP** - agregar OrdenEnDisputa es crear OrdenEnDisputa implements IEstadoOrden. Agregar PoliticaPreciosFestivos es crear nueva clase sin modificar SvcCalculoPrecio.
- **LSP** - cualquier IEstadoOrden o IPoliticaPrecios es intercambiable sin romper el comportamiento del agregado.
- **ISP** - IOrdenCommandRepository e IOrdenQueryRepository separados. Los UseCases de comando solo dependen de IOrdenCommandRepository.
- **DIP** - Orden depende de IEstadoOrden. DefCombo depende de IReglaCombo. SvcCalculoPrecio depende de IPoliticaPrecios. Nunca de clases concretas.

BC: Financiero

Transacción es el Agregado Raíz y es completamente inmutable una vez registrada. No conoce entidades de otros contextos (ni Pelicula, ni Sala, ni Combo). Solo trabaja con conceptos facturables abstractos. Para correcciones se emite una Transacción de reversión; nunca se modifica la original.

Entidad	Atributos clave	Tipo en DDD
Transaccion	id, ordenRef, conceptos[], descuentos[], valorTotal, timestamp, estado: IEstadoTransaccion	AggregateRoot (inmutable)
TransaccionReversion	id, transaccionOriginalRef, motivo, timestamp	Entity (en Transaccion)

Value Objects del BC:

Value Object	Definición y restricciones
TransaccionId	UUID inmutable. También sirve como idempotency key: si llega la misma orden dos veces, la segunda se rechaza con 422.
OrdenDepurada	Modelo propio del BC. No reutiliza el modelo de Ventas. Contiene solo: idOrden, conceptosFacturables[], descuentosAplicados[], valorTotal.

ConceptoFacturable	Descripcion (string) + valor (Dinero). Inmutable. Representa un item individual de la orden sin referencia a objetos del dominio cinematografico.
ReferenciaExterna	Identificador de la pasarela de pago externa. Inmutable una vez registrado. Trazabilidad hacia el sistema de pagos.
RegistroContable	Consolidacion contable final. Inmutable. Contiene: idTransaccion, fecha, valorTotal, estadoPago, referenciaExterna.

Elemento	Tipo	Responsabilidad
IEstadoTransaccion	Interfaz	registrar(), revertir(), esInmutable()
TransaccionPendiente	Clase	Permite registrar(). esInmutable() = false
TransaccionRegistrada	Clase	Permite revertir(). esInmutable() = true
TransaccionRevertida	Clase	Rechaza cualquier operación. esInmutable() = true

Elemento	Tipo	Responsabilidad
IPasarelaPago	Interfaz	procesar(ordenDepurada, metodoPago): EstadoPago, reintentar(idTransaccion): EstadoPago
PasarelaPSE	Clase	Procesamiento via PSE
PasarelaTarjeta	Clase	Procesamiento via tarjeta crédito/débito
PasarelaEfectivo	Clase	Procesamiento via pago en efectivo

Patrones aplicados:

- **State** - IEstadoTransaccion con TransaccionPendiente, TransaccionRegistrada, TransaccionRevertida.
- **Strategy** - IPasarelaPago con PasarelaPSE, PasarelaTarjeta, PasarelaEfectivo.

Principios SOLID aplicados:

- **SRP** - Transaccion gestiona el registro contable; IEstadoTransaccion gestiona las transiciones de estado; IPasarelaPago gestiona la comunicación con la pasarela externa; SvcRegistroContable gestiona la generación del RegistroContable.
- **OCP** - agregar PasarelaPayPal es crear PasarelaPayPal implements IPasarelaPago sin modificar SvcProcesoPago ni Transaccion. Agregar TransaccionSuspendida es crear una nueva clase sin modificar Transaccion.
- **LSP** - cualquier IPasarelaPago es intercambiable en SvcProcesoPago sin romper su comportamiento. Cualquier IEstadoTransaccion es intercambiable en Transaccion.
- **ISP** - ITransaccionCommandRepository e ITransaccionQueryRepository separados. Los UseCases de comando solo dependen del repositorio de comando.
- **DIP** - Transaccion depende de IEstadoTransaccion. SvcProcesoPago depende de IPasarelaPago. SvcRegistroContable depende de ITransaccionQueryRepository. Nunca de clases concretas.

Principio de aislamiento: El BC Financiero aplica aislamiento estricto: no recibe objetos del dominio como Pelicula, Sala ni ComboConfiteria. Solo recibe la OrdenDepurada con conceptos facturables y el valor total. Este principio garantiza que el BC Financiero pueda operar, auditarse y eventualmente externalizarse sin acoplamiento al dominio cinematografico.

BC: Cadena

Sucursal es el Agregado Raíz. Representa una instancia física del Multiplex dentro de la cadena corporativa. Cambia con muy poca frecuencia (configuración inicial y cambios organizacionales).

ContratoCorporativo es una entidad dentro de Sucursal. Representa acuerdos vigentes con terceros. Tiene su propio ciclo de vida (vigencia, condiciones).

ConfiguracionGlobal es una entidad embebida en Sucursal. Contiene parámetros organizacionales (zona horaria, moneda, reglas globales) que los demás servicios consultan al arrancar.

Entidad	Atributos clave	Tipo en DDD
---------	-----------------	-------------

Sucursal	id, nombre, configuracion: ConfiguracionGlobal, contratos[]	AggregateRoot
ContratoCorporativo	id, tercero, vigencia: Vigencia, condiciones, estado: IEstadoContrato	Entity (en Sucursal)
ConfiguracionGlobal	zonaHoraria, moneda, parametros[]	Entity embebida en Sucursal

Value Objects del BC:

Value Object	Definición y restricciones
SucursalId	UUID unico por sucursal. Inmutable.
Vigencia	FechaInicio + fechaFin. Compartido entre ContratoCorporativo y ConfiguracionGlobal. Invariante: fechaInicio < fechaFin.
ParametroGlobal	Clave (string) + valor (string) + tipo (STRING, ENTERO, BOOLEANO, DECIMAL). Value Object dentro de ConfiguracionGlobal. Evita un mapa de strings sin tipado.
NombreSucursal	String con longitud entre 3 y 100 caracteres. Unico dentro de la cadena.

Elemento	Tipo	Responsabilidad
IEstadoContrato	Interfaz	verificarVigencia(): boolean, cancelar(motivo): void, estaVigente(): boolean
ContratoVigente	Clase	verificarVigencia() evalúa Vigencia.fechaFin vs timestamp actual. estaVigente() = true. Permite cancelar()

ContratoVencido	Clase	estaVigente() = false. Rechaza cancelar(). No puede reactivarse automáticamente
ContratoCancelado	Clase	estaVigente() = false. Rechaza todas las operaciones

Patrones aplicados:

- **State** - IEstadoContrato con ContratoVigente, ContratoVencido, ContratoCancelado.
- **Observer** - ConfiguracionActualizada notifica a todos los contextos suscritos via IEventPublisher cuando cambia la ConfiguracionGlobal.

Principios SOLID aplicados:

- **SRP** - Sucursal gestiona la estructura organizacional; IEstadoContrato gestiona el ciclo de vida del contrato; ConfiguracionGlobal gestiona los parámetros organizacionales; SvcPropagacionConfiguracion gestiona la notificación de cambios.
- **OCP** - agregar ContratoSuspendido es crear ContratoSuspendido implements IEstadoContrato sin modificar ContratoCorporativo ni Sucursal. Agregar un nuevo ParametroGlobal es agregar una instancia sin modificar ConfiguracionGlobal.
- **LSP** - cualquier IEstadoContrato es intercambiable en ContratoCorporativo sin romper el comportamiento de Sucursal.
- **ISP** - ISucursalCommandRepository e ISucursalQueryRepository separados. Los UseCases de consulta solo dependen de ISucursalQueryRepository.
- **DIP** - ContratoCorporativo depende de IEstadoContrato. Sucursal depende de IEventPublisher para propagar cambios. Nunca de clases concretas.

Reglas de integridad:

- Ningun Multiplex puede modificar los ParametrosGlobales definidos por la cadena; solo puede leerlos.
- Un ContratoCorporativo debe estar en estado VIGENTE para que un servicio corporativo pueda ejecutarse.
- Una Sucursal no puede operar sin ConfiguracionGlobal asignada.
- La frecuencia esperada de cambio es maximo una vez por mes en condiciones normales.

EVENTOS DE DOMINIO

BC: Clientes y Fidelización

Evento	Disparado por	Datos	Consumidores
EspectadorRegistrado	Espectador.registrar()	idEspectador, nivel: NORMAL, timestamp	Ventas, Cadena
MembresiaActivada	Suscripcion.activar()	idEspectador, nivel, fechaInicio, fechaFin	Ventas
NivelAscendido	Suscripcion.ascender()	idEspectador, nivelAnterior, nivelNuevo, timestamp	Ventas
NivelDescendido	Suscripcion.descender()	idEspectador, nivelAnterior, nivelNuevo, timestamp	Ventas
MembresiaCancelada	Suscripcion.cancelar()	idEspectador, motivo, timestamp	Ventas

BC: Programación

Eventos	Disparado por:	Datos	Consumidor
PeliculaRegistrada	Pelicula.registrar()	IdPelicula, titulo, clasificación, genero, duración	Cartelera
FuncionProgramada	Funcion.programar()	IdFuncion, pelicula, sala, horario, formato	Infraestructura, cartelera
FuncionIniciada	Funcion.iniciar()	IdFuncion, sala, timestamp	infraestructura
FuncionFinalizada	Funcion.finalizar()	IdFuncion, sala, timestamp	Infraestructura, ventas

FuncionCancelada	Funcion.cancelar()	IdFuncion, sala, motivo, timestamp	Infraestructura
ActualizarCartelera	Actualizar.cartelera()	IdCartelera, funciones[], periodo	Ventas

BC: Infraestructura

Evento	Disparado por	Datos	Consumidores
SillaReservada	Sala.reservarSilla()	idSilla, idFuncion, idOrden, timestamp, expiracion	Ventas
ReservaRechazada	Sala.reservarSilla()	idSilla, idFuncion, idOrden, motivo: OCUPADA, timestamp	Ventas
SillaOcupada	Sala.ocuparSilla()	idSilla, idFuncion, timestamp	Ventas
SillaLiberada	Sala.liberarSilla()	idSilla, idFuncion, motivo: EXPIRACION/CANCELACION, timestamp	Ventas
ReservaExpirada	Scheduler (al alcanzar ReservaExpiracion)	idSilla, idOrden, timestamp	Ventas
SalaEnMantenimiento	Sala.enviarMantenimiento()	idSala, timestamp	Programación
SalaReactivada	Sala.reactivar()	idSala, timestamp	Programación

BC: Ventas

Evento	Disparado por	Datos	Consumidores
OrdenCreada	Orden.crear()	idOrden, idEspectador, items[], descuento, total, expiracion	Infraestructura (reservar sillas)
OrdenConfirmada	Orden.confirmar()	idOrden, conceptosFacturables[], descuentosAplicados[], valorTotal	Financiero (via Service Bus)

OrdenExpirada	Scheduler (al alcanzar Expiracion)	idOrden, idEspectador, sillaRefs[], timestamp	Infraestructura (liberar sillas)
OrdenCancelada	Orden.cancelar()	idOrden, idEspectador, sillaRefs[], motivo, timestamp	Infraestructura (liberar sillas)
StockAgotado	ProductoConfiteria via SvcGestionStock	idProducto, nombre, timestamp	Administrador
StockReducido	ProductoConfiteria via SvcGestionStock	idProducto, cantidadAnterior, cantidadNueva, timestamp	Interno BC Ventas

BC: Financiero

Evento	Disparado por	Datos que lleva	Consumidores
TransaccionRegistrada	Transaccion.registrar()	idTransaccion, idOrden, valorTotal, timestamp: REGISTRADA	Ventas
PagoAprobado	Pasarela externa → Transaccion.procesarPago()	idTransaccion, idOrden, referenciaExterna, timestamp	Ventas
PagoRechazado	Pasarela externa → Transaccion.procesarPago()	idTransaccion, idOrden, motivo, timestamp	Ventas
TransaccionRevertida	Transaccion.revertir()	idTransaccionOriginal, idTransaccionReversion, motivo, timestamp	Área contable

BC: Cadena

Evento	Disparado por	Datos	C
ConfiguracionActualizada	Sucursal.actualizarConfiguracion()	idSucursal, parametrosModificados[], timestamp	T c S
ContratoCorporativoRegistrado	Sucursal.registrarContrato()	idContrato, tercero, vigencia, timestamp	V
ContratoCorporativoVencido	Scheduler (al alcanzar fechaFin de Vigencia)	idContrato, idSucursal, timestamp	V
ContratoCorporativoCancelado	Sucursal.cancelarContrato()	idContrato, motivo, timestamp	V

REGLAS DE NEGOCIO E INVARIANTES

Lógica del Dominio o Lógica de Negocio Compleja

BC: Clientes y Fidelización

Un espectador no puede registrarse dos veces con el mismo Documento, el agregado rechaza la creación si el Documento ya existe.

Una Suscripcion no puede tener fechaFin anterior a fechaInicio el Value Object Vigencia valida esto en construcción.

Un espectador con Suscripcion CANCELADA tiene PorcentajeDescuento = 0.0 porque SuscripcionCancelada retornan 0.0 desde su implementación de IEstadoSuscripcion.

El nivel solo puede modificarse mediante Ascender() o Descender(). La validación de si puede ascender o descender la resuelve nivel.puedeAscender() y nivel.puedeDescender() — el agregado no evalúa el tipo concreto de nivel.

Un espectador en nivel PLATINO no puede ascender. Un espectador en nivel NORMAL no puede descender.

BC: Programación

Una Función no puede programarse en una Sala cuyo RangoHorario se solape con otra Función ya existente. El agregado Función valida esto antes de persistirse.

Una Película solo puede aparecer en Cartelera si tiene Clasificacion y FormatoProyeccion definidos. invariante del agregado Cartelera.

Una Función en estado FINALIZADA o CANCELADA tiene estado.esModificable() = false. Cualquier operación es rechazada por IEstadoFuncion sin que Funcion evalúe el tipo concreto de estado.

La Duración de una Película debe ser mayor a 0 y menor o igual a 300 minutos invariante del Value Object Duracion.

Una Función no puede programarse con RangoHorario en el pasado. invariante del Value Object RangoHorario.

BC: Infraestructura

Una Silla solo puede reservarse si su estado es DISPONIBLE; cualquier intento sobre RESERVADA u OCUPADA se rechaza con conflicto, esta es la invariante más crítica del sistema, tasa de doble venta = 0%.

La transición de estado de una Silla debe ejecutarse en una sola transacción atómica; no puede haber estados intermedios visibles.

Una Sala en EN_MANTENIMIENTO no puede tener Sillas en estado RESERVADA u OCUPADA.

El Aforo de una Sala es inmutable tras su creación, invariante del Value Object Aforo.

El número de Sillas activas en una Sala no puede superar el Aforo, el agregado Sala valida esto al agregar sillas.

Una ReservaExpiracion no puede ser un timestamp pasado en el momento de reservar, invariante del Value Object ReservaExpiracion.

tipo.esCompatibleConFormato(formato) debe retornar true. Salalmax implementa esta regla internamente — ni Sala ni Funcion evalúan tipos concretos.

tipoSilla.esCompatibleConTipoSala(sala.getTipo()) debe retornar true. SillaVip y SillaAcompañante implementan esta regla internamente.

Una Sala no puede salir de EN_MANTENIMIENTO si tiene Sillas en estado OCUPADA.

BC: Ventas

Una Orden debe tener al menos un ítem (ItemBoleta o ItemConfiteria) para poder crearse. invariante del agregado Orden.

El Descuento se calcula una sola vez al crear la Orden y queda congelado en el Value Object; cambios posteriores en el nivel del espectador no afectan órdenes existentes.

El precio del Combo se congela en el momento de agregarlo a la Orden; cambios en DefCombo.precioEspecial no afectan órdenes en curso.

No se puede agregar un ItemConfiteria si ProductoConfiteria.stock es 0, invariante del agregado Orden al agregar ítems.

estado.esModificable() retorna false en OrdenConfirmada, OrdenExpirada y OrdenCancelada. Cualquier operación es rechazada por IEstadoOrden sin que Orden evalúe el tipo concreto de estado.

Una Orden no puede tener dos ItemBoleta con la misma combinación de FuncionRef + SillaRef.

Un ItemBoleta de tipo VIP solo puede asociarse a una SillaRef de tipo VIP, invariante del agregado Orden al agregar ItemBoleta.

Una boleta solo puede crearse para una Función cuyo horario no haya pasado.

regla.esValido(precioEspecial, sumaProductos) evalúa esta condición. DefCombo no implementa la regla directamente — delega a IReglaCombo

Las boletas de evento corporativo tienen Descuento con porcentaje = 0.0 independientemente del nivel del espectador.

BC: Financiero

Una transacción registrada es completamente inmutable; ninguna operación puede modificar sus atributos una vez persistida.

Si la misma OrdenDepurada llega dos veces (mismo OrdenId), la segunda se rechaza con error de idempotencia, TransaccionId actúa como idempotency key.

Una Transacción no puede registrarse con valorTotal menor a 0.

Para correcciones se emite una TransaccionReversion; nunca se modifica la transacción original.

Un pago rechazado no genera RegistroContable pero sí genera evento PagoRechazado.

BC: Cadena

Ningún Multiplex puede modificar los ParametrosGlobales definidos por la cadena; solo puede leerlos, invariante del agregado Sucursal.

Una Sucursal no puede operar sin ConfiguracionGlobal asignada.

Un ContratoCorporativo no puede tener fechaFin anterior a fechaInicio, invariante del Value Object Vigencia.

Un ContratoCorporativo no puede activarse si ya existe otro contrato VIGENTE con el mismo tercero en el mismo RangoHorario.

Lógica Aplicativa o de Coordinación

Reserva de sillas en compra: cuando Ventas crea una Orden con ItemBoleta, coordina con Infraestructura para reservar cada silla antes de confirmar la creación. Si alguna silla falla, toda la Orden se rechaza, no se acepta reserva parcial. Implementado en el Servicio de Dominio de Ventas.

Consulta de descuento al crear Orden: Ventas coordina con Clientes para obtener el PorcentajeDescuento del espectador antes de calcular el total. Si Clientes no responde se aplica circuit breaker y la Orden no puede crearse. Implementado en el Servicio de Dominio de Ventas.

Precio base de boleta: el Servicio de Dominio de Ventas consulta a Programación el FormatoProyeccion y a Infraestructura el TipoSilla para construir el precio base antes de pasarlo al agregado Orden. El agregado no hace llamadas externas, recibe los datos ya resueltos.

Descuento en eventos corporativos: el Servicio de Dominio de Ventas detecta si la orden contiene boletas corporativas y pasa Descuento con porcentaje = 0.0 al agregado Orden, independientemente del nivel del espectador.

Liberación de sillas ante Orden expirada o cancelada: cuando una Orden transiciona a EXPIRADA o CANCELADA, el evento OrdenExpirada u OrdenCancelada notifica a Infraestructura para liberar las SillaRefs asociadas. Infraestructura procesa la liberación de forma asíncrona.

Envío de OrdenDepurada a Financiero: cuando una Orden se confirma, el Servicio de Dominio de Ventas construye la OrdenDepurada extrayendo solo conceptos facturables y valores, y la envía a Financiero. Financiero no recibe la Orden completa.

Liberación de sillas ante FuncionCancelada: cuando Programación cancela una Función, el evento FuncionCancelada coordina con Infraestructura para liberar todas las sillas reservadas y con Ventas para cancelar todas las Órdenes que contengan ItemBoleta con esa FuncionRef.

Reducción de stock al confirmar Orden: cuando una Orden se confirma, el Servicio de Dominio de Ventas reduce el stock de cada ProductoConfiteria incluido en los ItemConfiteria. Si el stock llega a 0 se dispara el evento StockAgotado hacia el administrador.

Propagación de ConfiguracionGlobal: cuando Cadena actualiza la configuración de una Sucursal, el evento ConfiguracionActualizada se propaga a todos los contextos afectados. Cada contexto decide qué parámetros consume.

Validación de contrato corporativo: al registrar un evento corporativo, el Servicio de Dominio de Ventas verifica en Cadena que exista un ContratoCorporativo vigente con el tercero solicitante y en Programación que la sala no tenga funciones regulares en el mismo RangoHorario.

SERVICIOS DE DOMINIO

BC: Clientes y Fidelización

Servicio CalculoDescuento - Servicio interno del BC

Calcula el porcentaje de descuento aplicable según el nivel activo y el tipo de compra. Aunque la regla básica vive en el agregado, este servicio maneja casos especiales como descuentos adicionales por eventos especiales o promociones temporales configuradas en ParametrosGlobales.

Operaciones:

calcularDescuento(nivel: INivel, tipoCompra): PorcentajeDescuento

Servicio ReactivarSuscripcion

cuando un espectador con membresía EXPIRADA decide pagar para reactivarla. No pertenece al agregado porque requiere verificar el pago aprobado en Financiero antes de reactivar.

Operación:

reactivar(idEspectador, pagoAprobado): void

BC: Programación

Servicio ValidacionHorario - Servicio interno del BC

La validación de solapamiento de horarios entre funciones no pertenece a una sola Función sino que requiere consultar todas las funciones existentes en una sala. Esta lógica no puede vivir en el agregado Función porque este no tiene visibilidad sobre otras funciones.

Operaciones:

validarDisponibilidadSala(salaRef, rangoHorario, funcionesExistentes[]): boolean — retorna true si la sala está disponible en ese rango.

obtenerConflictos(salaRef, rangoHorario): Funcion[] — retorna las funciones que generarían conflicto.

Servicio GestionCartelera - Servicio interno del BC

Coordina la publicación y retiro de funciones en la Cartelera. No pertenece al agregado Cartelera porque requiere validar el estado de la Función antes de incluirla y coordinar la notificación a Ventas.

Operaciones:

publicarFuncion(idFuncion): void — verifica condiciones y agrega a Cartelera.

retirarFuncion(idFuncion, motivo): void — retira de Cartelera y dispara CarteleraActualizada.

BC: Infraestructura

Servicio GestionAforo - Servicio interno del BC

Calcula en tiempo real la ocupación de una sala para una función específica. No pertenece al agregado Sala porque requiere cruzar el estado de todas las sillas con las reservas activas para una función concreta.

Operaciones:

calcularOcupacion(idSala, idFuncion): int - invoca silla.estado.esOcupada() o silla.estado.esReservada() definidos en IEstadoSilla.

calcularDisponibles(idSala, idFuncion): int - retorna cantidad de sillas DISPONIBLES.

verificarAforo(idSala): boolean - retorna true si hay al menos una silla disponible.

Servicio LiberacionReservas - Servicio interno del BC

Gestiona la liberación automática de reservas expiradas. No pertenece al agregado Sala porque requiere evaluar múltiples salas y múltiples sillas en paralelo. Es invocado por el orquestador del sistema periódicamente.

Operaciones:

liberarReservasExpiradas(): SillaLiberada[] - evalúa todas las sillas en estado RESERVADA cuya ReservaExpiracion haya pasado y las libera, retornando los eventos generados.

Servicio CambioEstadoSala

coordina las transiciones de estado de la Sala verificando condiciones que requieren visibilidad sobre múltiples sillas simultáneamente

Operaciones:

enviarMantenimiento(idSala): void, reactivar(idSala): void

BC: Ventas

Servicio CalculoPrecio - Servicio interno del BC

Calcula el precio base de una boleta cruzando el TipoSilla con el FormatoProyeccion. Esta lógica no pertenece al agregado Orden porque requiere datos que vienen de Infraestructura y Programación. El servicio resuelve los datos externos y entrega el precio al agregado ya calculado.

Operaciones:

calcularPrecioBoleta(tipoSilla, formatoProyeccion, parametrosPrecio): Dinero

calcularPrecioCombo(defCombo, cantidades[]): Dinero

aplicarDescuento(subtotal, descuento): Dinero

Servicio CreacionOrden - Servicio interno del BC

Orquesta el proceso completo de creación de una Orden. No pertenece al agregado Orden porque requiere coordinar múltiples consultas externas antes de que el agregado pueda crearse en estado consistente.

Operaciones:

crearOrden(idEspectador, itemsBoleta[], itemsConfiteria[]): Orden - consulta descuento a Clientes, precio base a Servicio CalculoPrecio, reserva sillas en Infraestructura y crea el agregado Orden con todos los datos resueltos.

cancelarOrden(idOrden, motivo): void - transiciona la Orden a CANCELADA y coordina la liberación de sillas en Infraestructura.

Servicio ConfirmacionOrden - Servicio interno del BC

Orquesta la confirmación de una Orden. Coordina la reducción de stock en ProductoConfiteria y el envío de la OrdenDepurada a Financiero.

Operaciones:

confirmarOrden(idOrden): void - confirma el agregado Orden, reduce stock de cada ItemConfiteria, construye OrdenDepurada y la envía a Financiero.

construirOrdenDepurada(orden): OrdenDepurada - extrae solo conceptosFacturables y valores sin objetos del dominio cinematográfico.

Servicio ValidacionEventoCorporativo - Servicio interno del BC

Valida si una orden de evento corporativo puede ejecutarse. No pertenece al agregado Orden porque requiere consultar a Cadena y a Programación.

Operaciones:

validarEventoCorporativo(tercero, salaRef, rangoHorario): boolean - verifica contrato vigente en Cadena y ausencia de conflictos en Programación.

BC: Financiero

Servicio ProcesoPago - Servicio interno del BC

Coordina la comunicación con la pasarela de pago externa. No pertenece al agregado Transaccion porque involucra integración con infraestructura externa, exactamente el tipo de dependencia que no debe contaminar el dominio.

Operaciones:

procesarPago(ordenaDepurada, metodoPago): EstadoPago - envía la orden a la pasarela y retorna el resultado.

reintentarPago(idTransaccion): invoca pasarela.reintentar(idTransaccion) - la lógica de reintento vive en la implementación concreta de IPasarelaPago.

cambiarPasarela(pasarela: IPasarelaPago): void - permite cambiar la pasarela activa sin modificar el servicio.

Servicio RegistroContable - Servicio interno del BC

Genera el RegistroContable final tras un pago aprobado. No pertenece al agregado Transaccion porque puede requerir enriquecimiento con datos de configuración contable de Cadena.

Operaciones:

generarRegistro(idTransaccion): RegistroContable

consultarHistorial(fechaInicio, fechaFin): RegistroContable[]

BC: Cadena

Servicio PropagacionConfiguracion - Servicio interno del BC

Gestiona la propagación asíncrona de cambios de configuración a todos los contextos afectados. No pertenece al agregado Sucursal porque requiere conocer qué contextos deben ser notificados y cómo.

Operaciones:

propagarCambio(idSucursal, parametrosModificados[]): void - publica ConfiguracionActualizada para todos los contextos suscritos.

Servicio GestionContratos - Servicio interno del BC

Gestiona el ciclo de vida de los contratos corporativos incluyendo la verificación periódica de vencimientos.

Operaciones:

verificarVigencias(): ContratoCorporativoVencido[] - evalúa todos los contratos y vence los que superaron su fechaFin.

consultarContratosVigentes(idSucursal, tercero): ContratoCorporativo[]

DISEÑO POR CONTRATO (DbC)

BC: Clientes y Fidelización

Espectador.registrar(nombre, correo, documento)

Precondiciones:

nombre no puede ser nulo ni vacío, mínimo 2 caracteres.

correo debe cumplir formato RFC 5321.

documento debe tener tipo válido (CC, CE, PAS) y número no vacío.

No debe existir otro Espectador con el mismo documento en el sistema.

Postcondiciones:

Se crea con nivel = new NivelNormal() y estado = new SuscripcionActiva()

Se genera el evento EspectadorRegistrado.

PorcentajeDescuento inicial = 0.0 hasta que la membresía se active.

Invariantes:

El Espectador siempre tiene un documento único en el sistema.

El Espectador siempre tiene exactamente una Suscripcion activa o inactiva.

Suscripcion.ascender()

Precondiciones:

La Suscripcion debe estar en estado ACTIVA.

nivel.puedeAscender() debe retornar true.

El pago del plan superior debe haber sido aprobado por Financiero.

Postcondiciones:

La instancia de INivel se reemplaza por la del nivel superior. nivel.calcularDescuento() retorna el nuevo porcentaje.

PorcentajeDescuento se recalcula según el nuevo nivel.

Se genera el evento NivelAscendido con nivelAnterior y nivelNuevo.

fechaInicio de la nueva membresía queda registrado con timestamp del momento del ascenso.

Invariantes:

El nivel siempre es uno de: NORMAL, ORO, PLATINO.

PorcentajeDescuento siempre es coherente con el nivel activo.

Suscripcion.descender()

Precondiciones:

nivel.puedeDescender() debe retornar true.

La operación debe ser iniciada por el espectador voluntariamente o por no renovación del plan.

Postcondiciones:

El nivel transiciona al inmediatamente inferior (PLATINO→ORO, ORO→NORMAL).

PorcentajeDescuento se recalcula según el nuevo nivel.

Se genera el evento NivelDescendido con nivelAnterior y nivelNuevo.

Invariantes:

El nivel siempre es uno de: NORMAL, ORO, PLATINO.

PorcentajeDescuento siempre es coherente con el nivel activo.

Suscripcion.expirar()

Precondiciones:

La Suscripcion debe estar en estado ACTIVA.

La fechaFin debe ser menor o igual al timestamp actual.

Postcondiciones:

La Suscripcion transiciona a estado EXPIRADA.

PorcentajeDescuento se recalcula a 0.0.

Se genera el evento MembresiaExpirada con idEspectador, nivelAnterior y timestamp.

Invariantes:

Una Suscripcion EXPIRADA nunca aplica descuentos.

Una Suscripcion EXPIRADA puede reactivarse mediante pago; una CANCELADA no.

Suscripcion.cancelar(motivo)**Precondiciones:**

La Suscripcion debe estar en estado ACTIVA o EXPIRADA.

motivo no puede ser nulo ni vacío.

Postcondiciones:

La Suscripcion transiciona a estado CANCELADA. Estado irreversible.

PorcentajeDescuento se recalcula a 0.0.

Se genera el evento MembresiaCancelada con idEspectador, motivo y timestamp.

Invariantes:

Una Suscripcion CANCELADA nunca puede reactivarse - requiere nuevo registro.

Una Suscripcion CANCELADA siempre tiene PorcentajeDescuento = 0.0.

Servicio CalculoDescuento.calcularDescuento(nivel, tipoCompra)**Precondiciones:**

nivel debe ser una instancia válida de INivel.

tipoCompra no puede ser nulo.

La Suscripcion asociada debe estar en estado ACTIVA.

Postcondiciones:

Retorna un PorcentajeDescuento entre 0.0 y 1.0 coherente con el nivel activo.

Si tipoCompra es CORPORATIVO retorna siempre 0.0 independientemente del nivel.

Invariantes:

El resultado siempre es un decimal entre 0.0 y 1.0.

El cálculo es determinístico - los mismos inputs siempre producen el mismo output.

BC: Programación

Funcion.programar(peliculaRef, salaRef, rangoHorario, formato)

Precondiciones:

peliculaRef debe referenciar una Película con Clasificacion y FormatoProyeccion definidos.

salaRef debe referenciar una Sala en estado DISPONIBLE.

rangoHorario.inicio debe ser mayor al timestamp actual.

rangoHorario no debe solaparse con ninguna Función ya existente en la misma sala.

formato debe ser una instancia válida de IFormatoProyeccion.

formato.esCompatibleConSala(tipoSala) debe retornar true.

Si la sala es IMAX, el formato debe ser IMAX o 3D.

Postcondiciones:

Se crea la Función en estado PROGRAMADA.

Se genera el evento FuncionProgramada con idFuncion, salaRef y horario.

La Cartelera incluye la Función si su horario es futuro.

Invariantes:

Una sala nunca tiene dos funciones con horarios solapados.

Una Función siempre tiene peliculaRef, salaRef y rangoHorario definidos.

Funcion.cancelar(motivo)

Precondiciones:

estado.esModificable() debe retornar true. - no puede cancelarse una Función FINALIZADA o EN_CURSO.

El motivo no puede ser nulo ni vacío.

Postcondiciones:

La Función transiciona a estado CANCELADA. Estado inmutable desde este punto.

Se genera el evento FuncionCancelada con idFuncion, salaRef y motivo.

Infraestructura libera todas las sillas reservadas para esta función.

Ventas cancela todas las Órdenes con ItemBoleta que referencien esta función.

Invariantes:

Una Función FINALIZADA o CANCELADA nunca puede modificarse.

Funcion.iniciar()

Precondiciones:

estado.esModificable() debe retornar true y estado.iniciar() debe ser una operación válida para el estado actual.

El timestamp actual debe ser mayor o igual a rangoHorario.inicio.

La Sala asignada debe estar en estado DISPONIBLE.

Postcondiciones:

La Función transiciona a estado EN_CURSO.

Se genera el evento FuncionIniciada con idFuncion, salaRef y timestamp.

La Sala transiciona a estado OCUPADA en Infraestructura.

Invariantes:

Una Función EN_CURSO no puede cancelarse ni reprogramarse.

Funcion.finalizar()

Precondiciones:

estado.finalizar() debe ser una operación válida para el estado actual — solo FuncionEnCurso la permite.

El timestamp actual debe ser mayor o igual a rangoHorario.fin.

Postcondiciones:

La Función transiciona a estado FINALIZADA. Estado completamente inmutable.

Se genera el evento FuncionFinalizada con idFuncion, salaRef y timestamp.

La Sala transiciona a estado DISPONIBLE en Infraestructura.

Todas las Sillas de la función transicionan a DISPONIBLE.

Invariantes:

Una Función FINALIZADA nunca puede modificarse bajo ninguna circunstancia.

Pelicula.registrar(titulo, clasificacion, genero, duracion)

Precondiciones:

titulo no puede ser nulo ni vacío, mínimo 2 caracteres.

clasificacion debe ser un valor válido del enum: G, PG, PG13, R.

genero no puede ser nulo ni vacío.

duracion debe ser mayor a 0 y menor o igual a 300 minutos.

Postcondiciones:

Se crea el agregado Pelicula con los datos proporcionados.

Se genera el evento PeliculaRegistrada con idPelicula y atributos.

Invariantes:

Una Pelicula siempre tiene titulo, clasificacion, genero y duracion definidos.

La duracion siempre es mayor a 0 y menor o igual a 300.

Cartelera.actualizar(idFuncion, operacion)

Precondiciones:

idFuncion debe referenciar una Función existente.

Si operacion es AGREGAR: la Función debe tener Clasificacion definida y formato debe ser instancia válida de IFormatoProyeccion..

Si operacion es RETIRAR: la Función debe estar en la Cartelera actual.

operacion debe ser AGREGAR o RETIRAR.

Postcondiciones:

Si AGREGAR: la Función se incluye en la Cartelera y se genera CarteleraActualizada.

Si RETIRAR: la Función se elimina de la Cartelera y se genera CarteleraActualizada.

CarteleraActualizada solo se dispara si el cambio afecta funciones con horario futuro.

Invariantes:

La Cartelera solo contiene Funciones con Clasificacion y FormatoProyeccion definidos.

La Cartelera nunca contiene Funciones FINALIZADAS o CANCELADAS.

Servicio ValidacionHorario.validarDisponibilidadSala(salaRef, rangoHorario, funcionesExistentes[])

Precondiciones:

salaRef no puede ser nulo.

rangoHorario.inicio debe ser mayor al timestamp actual.

rangoHorario.inicio debe ser menor a rangoHorario.fin.

funcionesExistentes puede ser una lista vacía pero no nula.

Postcondiciones:

Retorna true si ninguna Función en funcionesExistentes tiene RangoHorario solapado con el propuesto.

Retorna false si existe al menos una Función con solapamiento.

Invariantes:

El resultado es determinístico — los mismos inputs siempre producen el mismo output.

Dos rangos se solapan si $\text{inicio1} < \text{fin2}$ AND $\text{inicio2} < \text{fin1}$.

BC: Infraestructura

Sala.reservarSilla(sillaId, idFuncion, idOrden, expiracion)

Precondiciones:

La Silla identificada por sillaId debe existir dentro del agregado Sala.

silla.estado.reservar() debe ser una operación válida para el estado actual — solo SillaDisponible la permite..

La Sala debe estar en estado DISPONIBLE, no EN_MANTENIMIENTO.

expiracion debe ser un timestamp futuro mayor al momento actual.

idOrden no puede ser nulo.

Postcondiciones:

El estado de la Silla transiciona a RESERVADA en una sola operación atómica.

ReservaExpiracion queda asignada con el valor de expiracion.

Se genera el evento SillaReservada con idSilla, idFuncion, idOrden y timestamp.

Invariantes:

Tasa de doble venta = 0%. Nunca dos órdenes pueden reservar la misma silla para la misma función.

Una Silla siempre tiene exactamente un estado: DISPONIBLE, RESERVADA u OCUPADA.

El número de sillas activas nunca supera el Aforo de la Sala.

Sala.liberarSilla(sillaId, motivo)

Precondiciones:

silla.estado.liberar() debe ser una operación válida - SillaReservada y SillaOcupada la permiten; SillaDisponible la rechaza.

motivo debe ser EXPIRACION, CANCELACION o FIN_FUNCION.

Postcondiciones:

El estado de la Silla transiciona a DISPONIBLE.

Se genera el evento SillaLiberada con idSilla, motivo y timestamp.

ReservaExpiracion queda nula.

Invariantes:

Una Silla DISPONIBLE no puede liberarse nuevamente.

Sala.enviarMantenimiento()

Precondiciones:

La Sala debe estar en estado DISPONIBLE.

Todas las sillas deben tener silla.estado.esLibre() = true. La evaluación la hace IEstadoSilla, no el agregado directamente.

Postcondiciones:

La Sala transacciona a estado EN_MANTENIMIENTO.

Se genera el evento SalaEnMantenimiento con idSala y timestamp.

Programación es notificada para no asignar nuevas funciones a esta sala.

Invariantes:

Una Sala EN_MANTENIMIENTO nunca tiene Sillas en estado RESERVADA u OCUPADA.

Sala.ocuparSilla(sillaId, idFuncion)

Precondiciones:

La Silla debe estar en estado RESERVADA.

idFuncion debe coincidir con la función para la que fue reservada.

La Sala debe estar en estado OCUPADA (función en curso).

Postcondiciones:

El estado de la Silla transiciona a OCUPADA.

Se genera el evento SillaOcupada con idSilla, idFuncion y timestamp.

Invariantes:

Una Silla solo puede ocuparse desde estado RESERVADA, nunca desde DISPONIBLE directamente.

Sala.reactivar()

Precondiciones:

La Sala debe estar en estado EN_MANTENIMIENTO.

No debe haber Sillas en estado inconsistente dentro de la Sala.

Postcondiciones:

La Sala transiciona a estado DISPONIBLE.

Se genera el evento SalaReactivada con idSala y timestamp.

Programación es notificada para poder asignar nuevas funciones a esta sala.

Invariantes:

Una Sala DISPONIBLE siempre puede recibir nuevas funciones.

Servicio LiberacionReservas.liberarReservasExpiradas()

Precondiciones:

El timestamp actual debe ser mayor a la ReservaExpiracion de las sillas evaluadas.

Solo el scheduler del sistema puede invocar esta operación.

Postcondiciones:

Todas las Sillas en estado RESERVADA cuya ReservaExpiracion haya pasado transicionan a DISPONIBLE.

Se genera un evento SillaLiberada por cada silla liberada con motivo EXPIRACION.

Se genera un evento ReservaExpirada por cada Orden asociada notificando a Ventas.

Invariantes:

Ninguna Silla permanece en estado RESERVADA después de que su ReservaExpiracion haya pasado.

El tiempo máximo de detección de expiración no supera el intervalo de ejecución del scheduler.

BC: Ventas

Orden.crear(espectadorRef, itemsBoleta[], itemsConfiteria[], descuento, expiracion)

Precondiciones:

espectadorRef debe referenciar un Espectador con Suscripcion ACTIVA o en su defecto descuento = 0.0.

items[] debe contener al menos un ItemBoleta o un ItemConfiteria.

Cada SillaRef en ItemBoleta debe estar en estado DISPONIBLE en Infraestructura.

Cada ProductoConfiteria en ItemConfiteria debe tener stock > 0.

expiracion debe ser un timestamp futuro.

descuento.porcentaje debe estar entre 0.0 y 1.0.

Postcondiciones:

Se crea la Orden en estado PENDIENTE.

El total se calcula y congela en el Value Object Dinero.

El descuento queda congelado. no puede modificarse después.

Se genera el evento OrdenCreada con idOrden, items[] y total.

Infraestructura reserva cada SillaRef referenciada en ItemBoleta.

Invariantes:

Una Orden siempre tiene al menos un ítem.

El descuento de una Orden nunca cambia después de su creación.

Una Orden PENDIENTE siempre tiene una expiracion futura en el momento de creación.

Orden.confirmar()

Precondiciones:

estado.confirmar() debe ser una operación válida — solo OrdenPendiente la permite.

La Orden no debe haber expirado, expiracion debe ser mayor al timestamp actual.

El pago debe haber sido aprobado por Financiero.

Postcondiciones:

IEstadoOrden transiciona a OrdenConfirmada. estado.esModificable() retorna false desde este punto.

Se genera el evento OrdenConfirmada con conceptosFacturables[] y valorTotal.

El stock de cada ProductoConfiteria se reduce según las cantidades de los ItemConfiteria.

La OrdenDepurada se envía a Financiero.

Invariantes:

Una Orden CONFIRMADA nunca puede modificarse ni cancelarse.

Toda Orden CONFIRMADA tiene un registro contable en Financiero.

Orden.cancelar(motivo)

Precondiciones:

estado.cancelar(motivo) debe ser una operación válida — solo OrdenPendiente la permite.

motivo no puede ser nulo ni vacío.

Postcondiciones:

La Orden transiciona a estado CANCELADA.

Se genera el evento OrdenCancelada con sillaRefs[] y motivo.

Infraestructura libera todas las SillaRefs asociadas.

Invariantes:

Una Orden CANCELADA nunca puede reactivarse.

DefCombo.activar()

Precondiciones:

DefCombo debe tener al menos un consumible en su lista.

regla.esValido(precioEspecial, sumaProductos) debe retornar true.

DefCombo debe estar en estado inactivo.

Postcondiciones:

DefCombo.activo transiciona a true.

El combo queda disponible para ser agregado a nuevas Órdenes.

Invariantes:

Un DefCombo activo siempre tiene precioEspecial menor a la suma de sus productos.

Un DefCombo siempre tiene al menos un consumible.

DefCombo.desactivar()

Precondiciones:

DefCombo debe estar en estado activo.

Postcondiciones:

DefCombo.activo transiciona a false.

Se genera el evento ComboDesactivado con idDefCombo, nombre y timestamp.

Las Órdenes existentes que ya contienen este Combo no se ven afectadas.

Invariantes:

Un DefCombo desactivado no puede agregarse a nuevas Órdenes.

Las Órdenes en curso que contienen el Combo mantienen el precio congelado al momento de su creación.

Servicio CalculoPrecio.calcularPrecioBoleta(tipoSilla, formatoProyeccion, parametrosPrecio)

Precondiciones:

tipoSilla debe ser un valor válido del enum TipoSilla.

formatoProyeccion debe ser un valor válido FormatoProyeccion.

parametrosPrecio no puede ser nulo ni vacío.

Postcondiciones:

Retorna un Value Object Dinero con el precio base calculado según la combinación tipoSilla + formatoProyeccion.

El resultado es determinístico - los mismos inputs siempre producen el mismo output.

Invariantes:

El precio base siempre es mayor a 0.

El precio nunca se calcula dentro del agregado Orden - siempre llega ya resuelto.

Servicio ValidacionEventoCorporativo.validarEventoCorporativo(tercero, salaRef, rangoHorario)

Precondiciones:

tercero no puede ser nulo ni vacío.

salaRef no puede ser nulo.

rangoHorario.inicio debe ser mayor al timestamp actual.

rangoHorario.inicio debe ser menor a rangoHorario.fin.

Postcondiciones:

Retorna true si existe un ContratoCorporativo VIGENTE con el tercero en Cadena Y la sala no tiene funciones regulares en el mismo RangoHorario en Programación.

Retorna false con motivo específico si alguna condición falla.

Invariantes:

Un evento corporativo nunca puede ejecutarse sin contrato vigente.

Una sala reservada para evento corporativo nunca tiene funciones regulares en el mismo RangoHorario.

BC: Financiero

Transaccion.registrar(ordenaDepurada, metodoPago)

Precondiciones:

ordenaDepurada.idOrden no debe existir ya en el sistema - garantía de idempotencia.

ordenaDepurada.valorTotal debe ser mayor a 0.

ordenaDepurada.conceptosFacturables no puede estar vacío.

metodoPago debe ser un valor válido.

OrdenDepurada es un modelo propio del BC Financiero que por definición solo contiene conceptosFacturables[], descuentos[] y valorTotal. Su estructura garantiza el aislamiento - no es posible incluir objetos del dominio cinematográfico en su definición.

Postcondiciones:

Se genera un TransaccionId único que actúa como idempotency key.

El estado inicial de la Transaccion es PENDIENTE hasta que la pasarela responda.

Tras aprobación se genera el evento TransaccionRegistrada con timestamp contable.

Se genera el RegistroContable inmutable.

Invariantes:

Una Transaccion registrada es completamente inmutable.

Nunca existen dos Transacciones con el mismo OrdenId.

El valorTotal de una Transaccion nunca es menor o igual a 0.

Transaccion.revertir(motivo)

Precondiciones:

La Transaccion debe estar en estado REGISTRADA con pago APROBADO.

motivo no puede ser nulo ni vacío.

Solo el área contable puede invocar esta operación.

Postcondiciones:

Se crea una TransaccionReversion con referencia a la transacción original, motivo y timestamp.

La Transaccion original permanece inmutable - nunca se modifica.

Se genera el evento TransaccionRevertida.

Invariantes:

La Transaccion original nunca se modifica en ninguna circunstancia.

Toda reversión siempre referencia una Transaccion original válida

BC: Cadena

Sucursal.actualizarConfiguracion(parametrosModificados[])

Precondiciones:

parametrosModificados no puede estar vacío.

Cada ParametroGlobal debe tener clave, valor y tipo válidos.

Solo la dirección corporativa puede invocar esta operación.

La Sucursal debe existir y tener ConfiguracionGlobal asignada.

Postcondiciones:

Los parámetros modificados se actualizan en ConfiguracionGlobal.

Se genera el evento ConfiguracionActualizada con idSucursal y parametrosModificados[].

El evento se propaga via Service Bus a todos los contextos afectados.

Invariantes:

Una Sucursal siempre tiene ConfiguracionGlobal asignada.

Ningún contexto externo puede modificar ParametrosGlobales directamente.

Sucursal.registrarContrato(tercero, vigencia, condiciones)

Precondiciones:

tercero no puede ser nulo ni vacío.

vigencia.fechaInicio debe ser mayor o igual al timestamp actual.

vigencia.fechaFin debe ser mayor a vigencia.fechaInicio.

No debe existir otro ContratoCorporativo VIGENTE con el mismo tercero en el mismo RangoHorario.

condiciones no puede estar vacío.

Postcondiciones:

Se crea el ContratoCorporativo en estado VIGENTE.

Se genera el evento ContratoCorporativoRegistrado con idContrato, tercero y vigencia.

Ventas es notificado para habilitar la venta de eventos corporativos con ese tercero.

Invariantes:

Un ContratoCorporativo siempre tiene fechaFin mayor a fechaInicio.

No pueden existir dos contratos VIGENTES con el mismo tercero en el mismo RangoHorario.

CAPAS DE ARQUITECTURA POR MICROSERVICIO

Capa de Presentación (API): expone los endpoints REST. Recibe peticiones HTTP, las transforma en comandos o queries y las delega a la capa de Aplicación. No contiene lógica de negocio. Solo maneja serialización, autenticación y códigos de respuesta HTTP.

Capa de Aplicación: orquesta los casos de uso. Coordina los agregados, servicios de dominio y eventos. No contiene reglas de negocio — solo coordina quién hace qué y en qué orden. Aquí viven los Servicios de Aplicación que invocan los Servicios de Dominio.

Capa de Dominio: es el núcleo. Contiene agregados, entidades, objetos de valor, servicios de dominio, eventos de dominio e interfaces de repositorio. No depende de ninguna otra capa — ni de base de datos, ni de frameworks, ni de APIs externas.

Capa de Infraestructura: implementa los detalles técnicos. Repositorios concretos, conexiones a base de datos, publicadores de eventos, clientes HTTP hacia otros microservicios y adaptadores externos. Implementa las interfaces definidas en el Dominio.

Microservicio: Clientes y Fidelización

Capa de Presentación:

ClientesController — endpoints REST: POST /v1/clientes/registro, GET /v1/clientes/{id}/descuento

SuscripcionController — endpoints REST: POST /v1/clientes/{id}/ascender, POST /v1/clientes/{id}/descender

Capa de Aplicación:

RegistrarEspectadorUseCase — orquesta la creación del agregado Espectador y publica EspectadorRegistrado.

ConsultarDescuentoUseCase — consulta el agregado Espectador y retorna PorcentajeDescuento.

AscenderNivelUseCase — verifica pago aprobado en Financiero y delega a Suscripcion.ascender().

DescenderNivelUseCase — delega a Suscripcion.descender().

Capa de Dominio:

Agregados: Espectador, Suscripcion

Value Objects: EspectadorId, Email, Documento, NombreCompleto, NivelSuscripcion, PorcentajeDescuento, BeneficioActivo

Servicios de Dominio: SvcCalculoDescuento

Eventos: EspectadorRegistrado, NivelAscendido, NivelDescendido, MembresiaExpirada, MembresiaCancelada

Interfaces: IEspectadorRepository, IEventPublisher

Capa de Infraestructura:

EspectadorRepository — implementación concreta con base de datos.

EventPublisher — publica eventos al Service Bus.

FinancieroHttpClient — consulta si el pago del plan fue aprobado.

Microservicio: Programación**Capa de Presentación:**

CartelepaController — GET /v1/programacion/cartelera, GET /v1/programacion/funcion/{id}

FuncionController — POST /v1/programacion/funciones, DELETE /v1/programacion/funciones/{id}

PeliculaController — POST /v1/programacion/peliculas

Capa de Aplicación:

ProgramarFuncionUseCase — valida disponibilidad de sala y crea la Función.

CancelarFuncionUseCase — cancela la Función y coordina notificaciones.

ConsultarCarteleraUseCase — retorna funciones disponibles con filtros.

RegistrarPeliculaUseCase — crea el agregado Película.

Capa de Dominio:

Agregados: Funcion, Pelicula, Cartelera

Value Objects: PeliculaRef, SalaRef, RangoHorario, FormatoProyeccion, Clasificacion, Duracion, PeriodoCartelera

Servicios de Dominio: SvcValidacionHorario, SvcGestionCartelera

Eventos: PeliculaRegistrada, FuncionProgramada, FuncionIniciada, FuncionFinalizada, FuncionCancelada, CarteleraActualizada

Interfaces: IFuncionRepository, IPeliculaRepository, ICarteleraRepository, IEventPublisher

Capa de Infraestructura:

FuncionRepository, PeliculaRepository, CarteleraRepository

EventPublisher

InfraestructuraHttpClient — consulta disponibilidad de sala.

Microservicio: Infraestructura Física

Capa de Presentación:

SalaController — GET /v1/infraestructura/salas/{id}/disponibilidad, PUT /v1/infraestructura/sillas/{id}/reservar

MantenimientoController — POST /v1/infraestructura/salas/{id}/mantenimiento, POST /v1/infraestructura/salas/{id}/reactivar

Capa de Aplicación:

ReservarSillaUseCase — recibe idSilla, idFuncion, idOrden y delega a Sala.reservarSilla().

LiberarSillaUseCase — recibe idSilla y motivo, delega a Sala.liberarSilla().

ConsultarDisponibilidadUseCase — retorna estado de todas las sillas de una sala para una función.

EnviarMantenimientoUseCase — delega a Sala.enviarMantenimiento().

ReactivarSalaUseCase — delega a Sala.reactivar().

Capa de Dominio:

Agregados: Sala, Silla

Value Objects: SalaId, SillaId, Posicion, Aforo, TipoSala, TipoSilla, ReservaExpiracion

Servicios de Dominio: SvcGestionAforo, SvcLiberacionReservas

Eventos: SillaReservada, ReservaRechazada, SillaOcupada, SillaLiberada, ReservaExpirada, SalaEnMantenimiento, SalaReactivada

Interfaces: ISalaRepository, IEventPublisher

Capa de Infraestructura:

SalaRepository

EventPublisher

ReservaExpirationScheduler — proceso de background que evalúa ReservaExpiracion periódicamente.

Microservicio: Ventas

Capa de Presentación:

OrdenController — POST /v1/ventas/orden, POST /v1/ventas/orden/{id}/confirmar, DELETE /v1/ventas/orden/{id}

ProductoController — GET /v1/ventas/productos, GET /v1/ventas/combos

Capa de Aplicación:

CrearOrdenUseCase — coordina consulta de descuento a Clientes, precio base, reserva de sillas en Infraestructura y crea la Orden.

ConfirmarOrdenUseCase — confirma la Orden, reduce stock y envía OrdenDepurada a Financiero.

CancelarOrdenUseCase — cancela la Orden y libera sillas en Infraestructura.

ConsultarProductosUseCase — retorna catálogo de ProductoConfiteria y DefCombo activos.

Capa de Dominio:

Agregados: Orden, ProductoConfiteria, DefCombo

Entidades: ItemBoleta, ItemConfiteria

Value Objects: OrdenId, EspectadorRef, FuncionRef, SillaRef, ProductoRef, DefComboRef, Dinero, Descuento, Expiracion, EstadoOrden, Combo

Servicios de Dominio: SvcCalculoPrecio, SvcCreacionOrden, SvcConfirmacionOrden, SvcValidacionEventoCorporativo

Eventos: OrdenCreada, OrdenConfirmada, OrdenExpirada, OrdenCancelada, StockAgotado, StockReducido, ComboDesactivado

Interfaces: IOrdenRepository, IProductoRepository, IDefComboRepository, IEventPublisher, ICientesClient, IInfraestructuraClient, IProgramacionClient, ICadenaClient

Capa de Infraestructura:

OrdenRepository, ProductoRepository, DefComboRepository

EventPublisher

CientesHttpClient, InfraestructuraHttpClient, ProgramacionHttpClient,
CadenaHttpClient

OrdenExpirationScheduler — proceso de background que evalúa Expiracion
periódicamente.

Microservicio: Financiero**Capa de Presentación:**

TransaccionController — POST /v1/financiera/transacciones, GET
/v1/financiera/transacciones/{id}

HistorialController — GET /v1/financiera/historial

Capa de Aplicación:

RegistrarTransaccionUseCase — recibe OrdenDepurada y coordina el procesamiento
del pago.

RevertirTransaccionUseCase — crea TransaccionReversion sin modificar la original.

ConsultarHistorialUseCase — retorna registros contables por rango de fecha.

Capa de Dominio:

Agregados: Transaccion

Entidades: TransaccionReversion

Value Objects: TransaccionId, OrdenDepurada, ConceptoFacturable, EstadoPago,
ReferenciaExterna, RegistroContable

Servicios de Dominio: SvcProcesoPago, SvcRegistroContable

Eventos: TransaccionRegistrada, PagoAprobado, PagoRechazado,
TransaccionRevertida

Interfaces: ITransaccionRepository, IEventPublisher, IPasarelaClient

Capa de Infraestructura:

TransaccionRepository

EventPublisher

PasarelaHttpClient — integración con pasarela de pago externa.

Microservicio: Cadena**Capa de Presentación:**

SucursalController — GET /v1/cadena/configuracion/{idSucursal}

ContratoController — POST /v1/cadena/contratos, DELETE /v1/cadena/contratos/{id}

Capa de Aplicación:

ConsultarConfiguracionUseCase — retorna ConfiguracionGlobal de una Sucursal.

ActualizarConfiguracionUseCase — actualiza ParametrosGlobales y coordina propagación.

RegistrarContratoUseCase — crea ContratoCorporativo y notifica a Ventas.

CancelarContratoUseCase — cancela ContratoCorporativo y notifica a Ventas.

Capa de Dominio:

Agregados: Sucursal

Entidades: ContratoCorporativo, ConfiguracionGlobal

Value Objects: SucursalId, Vigencia, EstadoContrato, ParametroGlobal, NombreSucursal

Servicios de Dominio: SvcPropagacionConfiguracion, SvcGestionContratos

Eventos: ConfiguracionActualizada, ContratoCorporativoRegistrado, ContratoCorporativoVencido, ContratoCorporativoCancelado

Interfaces: ISucursalRepository, IEventPublisher

Capa de Infraestructura:

SucursalRepository

EventPublisher

ContratoVigenciaScheduler — proceso de background que evalúa vencimientos de contratos.

ESTRUCTURA DE CARPETAS

```
Multiplex/
├─ ArquitecturaDDD-Multiplex.md      ← Documento maestro de diseño DDD
├─ Multiplex.slnx                    ← Solución (.NET 10)
├─ docker-compose.yml                ← Postgres + RabbitMQ + pgAdmin
├─ docker/
│   └─ postgres-init.sql              ← Crea las 6 BDs (multiplex_<bc>)
└─ src/
    ├─ Directory.Build.props          ← Settings comunes (LangVersion, Nullable)
    ├─ Shared/
    │   ├─ Shared.Kernel/              ← Entity, AggregateRoot, ValueObject, Result, Money, IDomainEvent
    │   └─ Messaging.Contracts/        ← Eventos de integración (records inmutables) por BC
    ├─ Cadena/
    │   ├─ Cadena.Domain/              ← Sucursal, ContratoCorporativo, ConfiguracionGlobal
    │   ├─ Cadena.Application/          ← ConsultarConfiguracion, RegistrarContrato, ...
    │   ├─ Cadena.Infrastructure/       ← DbContext, Repos, ContratoVigenciaScheduler
    │   └─ Cadena.Api/                 ← SucursalController, ContratoController
    ├─ Clientes/                      ← idem (Espectador + Suscripcion)
    ├─ Financiero/                    ← idem (Transaccion inmutable + StubPasarelaClient)
    ├─ Infraestructura/               ← idem (Sala + Silla + ReservaExpirationScheduler)
    ├─ Programacion/                 ← idem (Pelicula, Funcion, Cartelera, Alquiler)
    └─ Ventas/                       ← idem (Orden, ProductoConfiteria, DefCombo + OrdenExpirationSched
```